

AIGA – AI for Generation and Automation (Internship Program, Updated)

Executive Summary: The **AIGA** internship is an intensive 12-week (90-hour) program preparing B.Tech students for industry roles in generative AI and automation. It covers AI fundamentals (neural nets, NLP), advanced topics (transformers, LLMs, small language models), engineering practices (prompt/context/inference engineering, RAG pipelines), tool usage (Google AI Studio/Gemini, Antigravity, GitHub Copilot, etc.), and emergent skills (agents, AI ethics). Importantly, the curriculum now includes dedicated modules on **Evaluation & Metrics** (e.g. BLEU/ROUGE/BERTScore and LLM-as-judge ¹ ²), **Cost Awareness** (token counting, prompt optimization ³), **Responsible AI** (bias, privacy, hallucination mitigation, red-teaming ⁴), **Data Engineering for RAG** (document chunking, embedding strategy ⁵), and **Real-Time Streaming** (SSE/WebSockets) ⁶. Students also learn to train a **Small Language Model (SLM)** via fine-tuning open-source models ⁷ ⁸. The program balances lectures and hands-on labs, with daily 1.5-hour sessions (5 days/week) totaling ~90 hrs. Each day includes theory, coding exercises, or labs; every week has a capstone or project milestone. Assessment combines labs, quizzes, and projects evaluated by clear rubrics ⁹ ¹⁰. Industry partnerships (Google Cloud, IBM, Microsoft) support cloud credits, guest sessions, and eventual certification opportunities (e.g. Google ML Engineer, IBM AI Engineering). Infrastructure requirements (GPU cloud/AI API costs) are budgeted; for example, renting an A100 GPU costs ~\$0.78–5.78/hr ¹¹, and GPT-4-like models ~\$0.03–0.06 per 1K tokens input/output. Outcomes align with B.Tech Program Outcomes (fundamentals, problem analysis, design, tool usage, teamwork) ¹² ⁹.

Prerequisite Knowledge

Students should have **strong programming skills (Python)**, basic understanding of calculus/linear algebra, and prior coursework in introductory Machine Learning (classification, loss functions). Familiarity with Linux, Git, and data handling (Pandas, SQL) is assumed. This aligns with typical B.Tech curricula and sets the foundation for advanced AI topics.

Program Outcomes Mapping

Completion of AIGA maps to B.Tech program outcomes ¹²:

- **Engineering Knowledge (PO1):** AI/ML theory (neural nets, statistics) and modern tools.
- **Problem Analysis (PO2):** Framing AI tasks (e.g. RAG chatbot requirements) and diagnosing model behavior.
- **Design/Development (PO3):** Designing AI pipelines (transformer models, RAG architecture, agents) and software components.
- **Modern Tool Usage (PO5):** Using current AI platforms (Gemini API, Antigravity, Copilot, HuggingFace) and coding libraries.
- **Ethics (PO8) & Society:** Addressing AI fairness, bias, privacy, and societal impact in projects.
- **Teamwork & Communication (PO9,PO10):** Collaborative labs and presentations build these skills.

These outcomes are assessed through project deliverables, coding assignments, and presentations, using rubrics from educational internships ⁹ ¹⁰.

Syllabus & Learning Objectives

The syllabus covers key topics with module-level objectives:

- **Module 1 (Weeks 1–2): AI/ML Fundamentals.** Introduce artificial neural networks: perceptrons, activation functions, MLPs, and basic CNN/RNN architectures. Objectives include understanding backpropagation for training, and implementing simple NNs in PyTorch/TensorFlow. *Hands-on:* Build/train an MLP classifier and a CNN on a toy dataset.
- **Module 2 (Week 2): NLP Basics.** Introduce natural language processing: tokenization, text preprocessing, and embeddings. Topics: Word2Vec, GloVe, and transformer embeddings (BERT embeddings). Objectives: process text data, use pre-trained embeddings for classification. *Hands-on:* Train a sentiment classifier; practice tokenizing and embedding text.
- **Module 3 (Week 3): Transformer Architectures.** Study the Transformer model ¹³. Understand self-attention, multi-head attention, and positional encoding. *Illustration:* see Figure below (embedding + Q/K/V process). Objectives: explain how transformers capture context, and implement an attention layer. *Hands-on:* Visualize attention weights; code a simple transformer block.

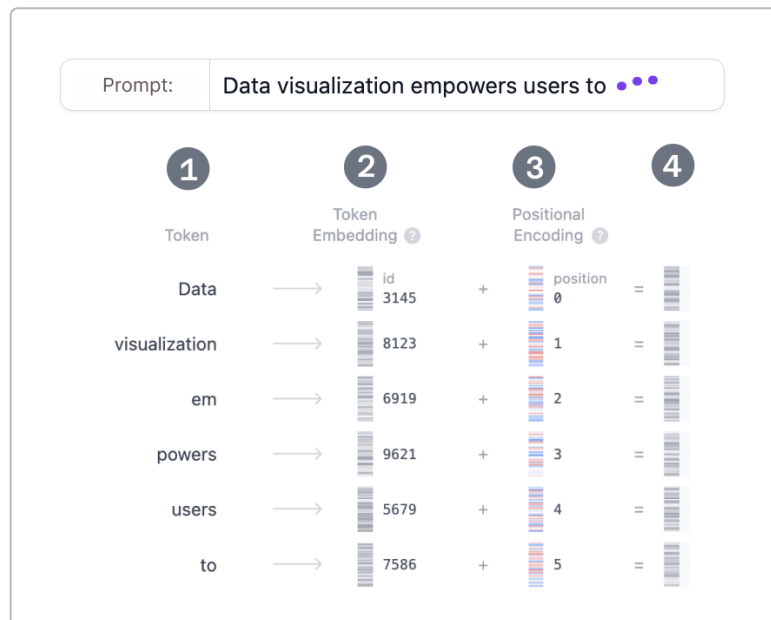


Figure: **Transformer Transformer (Embedding + Attention)** – Token embeddings (with positional encodings) feed into multi-head attention ¹³. Students build and test these modules.

- **Module 4 (Week 4): Large vs. Small Language Models.** Compare large pretrained LLMs (GPT/BERT) with **SLMs (Small LMs)**. Objectives: discuss scale, personal/custom LMs, and fine-tuning. *Case study:* Introduction to building a small language model (SLM) ⁷ – e.g. fine-tuning an open 7B model (Phi-2, Mistral, TinyLlama) to a domain/persona. Students will perform LoRA/Q-LoRA fine-tuning on ~100–500 examples ⁸. *Hands-on:* Fine-tune an SLM and generate sample outputs.
- **Module 5 (Weeks 5): Prompt Engineering & Cost Optimization.** Learn to craft effective prompts for LLMs ¹⁴ (zero-shot, few-shot, chain-of-thought techniques). Introduce system/user messages and template design. **Cost-awareness:** Estimate token usage with tools (OpenAI's tiktoken) and API pricing. For example, counting tokens with tiktoken helps predict cost ³. Objectives: minimize token use by prompt compression without losing quality. *Lab:* Given a task

(e.g. summarization), students design a prompt, count tokens, and iteratively refine it to reduce cost.

- **Module 6 (Week 6): Context Engineering & Data Prep.** Cover RAG pipelines: retriever + LLM. Focus on data engineering for RAG: document chunking (token-based, page-level, section-level) ⁵ and metadata tagging. Students learn strategies (e.g. 15% overlap chunks) ¹⁵ to maximize retrieval of relevant context. Discuss embedding choice (sentence vs paragraph embeddings) and index storage. *Lab:* Build a mini RAG: chunk a set of documents, generate embeddings (e.g. via Sentence-BERT), and implement a nearest-neighbor retrieval.
- **Module 7 (Week 7): Evaluation & Metrics.** Teach how to **evaluate RAG/chatbot performance**. Topics include automatic metrics: **BLEU** and **ROUGE** for n-gram overlap ¹ ¹⁶, and **BERTScore** for semantic similarity ². Discuss limitations (these capture surface overlap but not semantics ¹⁷). Introduce *LLM-as-judge* methods (e.g. G-Eval, GPTScore) where an LLM scores answers. RAG-specific metrics: Contextual Relevancy (how well the retriever fetched useful info) ¹⁸, and Answer Accuracy (agreement with ground truth). *Lab:* Evaluate outputs from their RAG system using BLEU/ROUGE and also a semantic scorer. Compare with manual evaluation, and refine prompt/context based on insights.
- **Module 8 (Week 8): Inference Engineering & Streaming.** Cover deploying models efficiently: latency/throughput tradeoffs, quantization, batching, and cache reuse. Include a **Streaming inference** lesson: SSE vs WebSocket for token streaming ⁶, to improve UX and reduce wait times. Objectives: implement a streaming API (e.g. OpenAI stream) in code and allow aborting to save tokens. *Lab:* Build a small web demo streaming LLM responses (use SSE) and measure time until first token (TTFT) improvement ⁶.
- **Module 9 (Week 9): AI Tools for Content & Code.** Hands-on use of platforms: **Google AI Studio (Gemini)** ¹⁹ for generative text/image/video; **GitHub Copilot** for coding assistance ²⁰; and **Google Antigravity** for agentic coding tasks ²¹ ²². *Lab:* Use Gemini API to generate a blog post and an image. Use Copilot to write a function, then launch Antigravity to create a small coding agent (e.g. auto-testing). Evaluate each tool's ease-of-use, cost (free tiers vs. paid), and integration in workflows.
- **Module 10 (Week 10): Agentic AI & Automation.** Study AI agents that autonomously perform tasks using LLMs and tools ²³ ²⁴. Components: agent planner (LLM with tools), tool APIs (search, calculators, custom functions), and multi-step workflows. Discuss frameworks (LangChain, ReACT). *Lab:* Build a simple agent (e.g. budget chatbot that uses web search and calculator tools). Demonstrate it planning and executing a task, then evaluate its **Task Completion** (did it achieve the goal).
- **Module 11 (Week 11): Responsible AI & Red-Teaming.** Deep dive on AI ethics. Topics: **Bias and Fairness** (recognizing and mitigating demographic biases), **Privacy** (avoiding private data leaks in prompts), **Hallucination** (false content risks). Introduce **Red-Teaming**: actively stress-testing the AI with adversarial prompts to find vulnerabilities ⁴. For example, attempt prompt-injection or toxic queries to test safeguards. *Lab:* Perform a red-teaming exercise: given an LLM prompt, iteratively craft inputs to elicit biased or harmful outputs. Record findings and discuss mitigations (filtering outputs, adding guardrails).
- **Module 12 (Week 12): Capstone Projects & Review.** Students work in teams on an industry-style capstone (e.g. a domain-specific RAG chatbot with an agentic front-end). Deliverables: a functional prototype, code repo, and presentation. This week also includes final reviews, feedback, and certification guidance.

Reading & Resources: Official docs and key papers supplement the modules (e.g. Vaswani et al. 2017 on Transformers ¹³, OpenAI GPT-3 blog ²⁵, LangChain docs for agents, Google Cloud prompt guide ¹⁴). Recommended hands-on tutorials include Hugging Face Transformer tutorials, tiktoken cost analysis ³, Nvidia's chunking blog ⁵, and the Antigravity and Gemini official guides ²¹ ²⁶. Assigned readings are given weekly (and links in resources).

Faculty & Mentorship

- **Instructors:** 1–2 faculty coordinating theory sessions.
- **Industry Mentors:** ~1 mentor per 8–10 students for labs and projects (total ~3 mentors for 25 students). Mentors guide project selection, code reviews, and reinforce best practices.
- **Roles:** Faculty handle lectures, evaluate concept quizzes; mentors supervise labs, grade assignments, and interact with students daily.
- **Student Teams:** Each team (4–5 students) has an assigned mentor to ensure manageable load and personalized guidance.

Infrastructure & Costs

- **Compute:** Provide GPU access (cloud or on-prem). For example, using Google Cloud Vertex AI or AWS:
- Rent ~NVIDIA A100: ~\$0.78–5.78/hr ¹¹ depending on tier.
- Even 200 GPU-hours (~25 sessions) can cover a course cohort.
- **Cloud APIs:** Leverage education/free tiers. Example: Google Gemini free trial (limited tokens) and free Vertex credits; OpenAI ChatGPT free tier or discounted rates for students.
- **Model Hosting:** Use open-source models for offline tasks to save cost (e.g. use local LLaMA in labs).
- **Budget:** Estimate \$2000–4000 total for cloud GPU/API usage (assuming some GPU hours and a few thousand tokens of paid LLM usage).
- **Licenses:** GitHub Copilot is free for students ²⁷ ; otherwise need organizational license (\$10/mo typical). All other tools have free or academic licenses.

Evaluation & Assessment

- **Daily Labs:** Short coding or design exercises are given each day, reinforcing the topic. Instructors or mentors grade these promptly for feedback.
- **Weekly Assignments:** Each week has a mini-project or problem set (e.g. “implement a prompt-ranking system” or “write a report on chunking strategy”) to be submitted by week’s end. These are graded on correctness and depth.
- **Capstone Project:** Teams work on a semester-long project with defined milestones (see schedule). Evaluation rubric includes: functionality, innovation, use of AI techniques, teamwork, documentation, and presentation quality ⁹ ¹⁰ . For example, rubrics emphasize *technical correctness*, *quality of solution*, *report clarity*, and *team collaboration*.
- **Quizzes & Exams:** Short quizzes may test theory (e.g. writing out BLEU formula, explaining attention).
- **Peer Review:** Teams may peer-review other projects (code review, critique) to foster collaborative learning.
- **Metrics-based Evaluation:** Student projects using RAG are also evaluated via the metrics taught (BLEU/ROUGE, LLM-judge) as part of learning the evaluation process.

Tools/Platform Comparison

Platform	Features	Pros	Cons	Cost (approx)
Google AI Studio (Gemini API) ¹⁹ ²⁶	Multimodal LLM (text, image, video); 1M-token context; function calling & built-in tools (search, code) ²⁸	State-of-the-art models (Gemini 3.1 Pro), Google ecosystem, free trial credits	Limited outside GCP, evolving API; free tier quotas	Free start (Vertex AI credits); pay-as-you-go thereafter
Google Antigravity ²¹ ²²	Agentic dev environment: AI-assisted IDE + agent manager; supports multiple LLMs (Gemini, Claude, GPT-OSS) ²²	Enables planning/ executing tasks in code; free preview for individuals ²²	Pre-alpha, limited docs, Windows support via CLI only ²²	Free (preview)
OpenAI (ChatGPT/API)	GPT-4o/LLaMA-like chatbot; plugins; LLM-as-API; strong few-shot performance	High-quality outputs, extensive docs & community; easy chat interface	Paid usage (per token); privacy concerns; less control over models	Free tier; ChatGPT Plus ~\$20/mo; API ~\$0.03/1K tokens (input)
GitHub Copilot ²⁰	AI coding assistant (IDE plugin); code/ chat suggestions; Copilot Chat/ Spaces	Seamless coding autocomplete; free for students; supports multi-language	Limited to code tasks; cloud-based (privacy?), subscription outside edu	Free for verified students; \$10/mo individual otherwise
Hugging Face (Transformers/ Spaces)	Vast open models (LLaMA, Falcon, etc.); hosted APIs and collaborative notebooks	Wide model choice; open-source; free tiers; strong community	Model quality may lag proprietary; requires more setup for large models	Free community tier; paid plans for heavy use or HF Inference endpoints
LangChain/ Agent frameworks	Toolkit for building LLM apps and agents with chains, memories	Powerful for prototyping agents with tools; wide ecosystem	Not standalone service; requires coding	Open-source (free); hosting costs apply
IBM watsonx / Azure AI	Enterprise AI (chatbots, vision, etc.); orchestration tools like watsonx Orchestrate	Enterprise features (gov compliance, data governance)	Focus on enterprise; less accessible to students; paid service	Various: free tier credits, paid usage per service

Day-wise Schedule (5 days/week, 1.5 hr/day, 90 mins each)

Week 1 (9 hrs): AI/ML Foundations

- Day 1: **Neural Nets Intro:** Perceptron, activation functions. **Daily Assignment:** Implement a sigmoid neuron for AND/OR.
- Day 2: **Feedforward NN:** Multi-layer perceptron; backprop algorithm. *Assignment:* Hand-trace 2-layer NN on simple data.
- Day 3: **CNN/RNN Basics:** Convolution and pooling intuition; simple RNN cell. *Lab:* Build a 1D CNN for text classification (Keras).
- Day 4: **Lab:** Code a small neural net (PyTorch) on MNIST or similar. *Weekly Assignment:* Write a report on training curves (loss/accuracy).
- Day 5: **NLP Overview:** Tokenization, BoW, TF-IDF. *Quiz:* Short quiz on NN terminology; *Assignment:* Preprocess a text corpus.

Week 2 (9 hrs): NLP & Embeddings

- Day 6: **Word Embeddings:** Word2Vec, GloVe. *Lab:* Generate 2D plot of word vectors.
- Day 7: **Contextual Embeddings:** BERT, sentence embeddings. *Assignment:* Extract BERT embedding for sample sentences.
- Day 8: **Text Classification:** LSTM/Transformer for sentiment analysis. *Lab:* Fine-tune a small transformer (HuggingFace) on a movie review dataset.
- Day 9: **Sequence Tasks:** Q&A and translation examples. *Assignment:* Evaluate model on custom test sentences.
- Day 10: **Quiz/Discussion:** Key NLP concepts; *Weekly Assignment:* Build and test an NER (named entity recognition) pipeline.

Week 3 (9 hrs): Transformers & LLM Overview

- Day 11: **Transformer Architecture:** Self-attention, Q/K/V, multi-head ¹³. *Illustration:* embedding+attention figure above.
- Day 12: **Attention Demo:** Visualize attention heads; explain their role in context. *Lab:* Use a transformer library to show attention heatmaps.
- Day 13: **Pretraining vs Finetuning:** GPT vs BERT paradigms; LLM capabilities (text, code). *Assignment:* Summarize GPT-3 findings (few-shot learning) ²⁵.
- Day 14: **Lab:** Using a pretrained transformer, generate text completions. Experiment with prompt variations.
- Day 15: **Introduction to RAG/SLM:** Show a simple RAG workflow. Introduce concept of Small LMs (SLM) and personalization ²⁹. *Weekly Assignment:* Describe potential use-case for a custom SLM.

Week 4 (9 hrs): Small LM (SLM) Training

- Day 16: **SLM Overview:** Use-cases for personal/custom models ²⁹.
- Day 17: **Base Model Selection:** Choose an open model (Phi-2, Mistral, TinyLlama) ⁷. Discuss local vs cloud training.
- Day 18: **Fine-Tuning Techniques:** LoRA/QLoRA explained ⁸. Gather/sample training data. *Assignment:* Collect 100–200 text examples for fine-tuning.
- Day 19: **Lab:** Perform LoRA fine-tuning on chosen SLM (50–100 samples); monitor training progress.
- Day 20: **Inference & Demo:** Generate text from SLM; compare style to original model. *Weekly Assignment:* Write a 1-page reflection on SLM strengths/limits.

Week 5 (9 hrs): Prompt Engineering & Cost

- Day 21: **Prompt Basics:** Zero-shot vs few-shot; system/user roles. *Lab:* Craft a prompt for GPT to summarize news, observe output.

- Day 22: **Advanced Prompting:** Chain-of-thought prompts; instruction formats. *Assignment:* Try CoT for math reasoning tasks.
- Day 23: **Prompt as Context:** Context length limits. **Cost:** Introduce token billing. *Quiz:* Calculate tokens manually for given text.
- Day 24: **Token Counting Lab:** Use tiktoken to count tokens of prompts ³. Estimate cost with sample API rates (e.g., \$0.03 per 1K tokens).
- Day 25: **Optimization Lab:** Given a prompt, iteratively shorten it (e.g. remove stopwords, system messages) and observe cost savings vs performance. *Weekly Assignment:* Optimize a long prompt for conciseness.

Week 6 (9 hrs): Context Engineering & Data Prep

- Day 26: **RAG Pipeline:** Components: retriever (vector search) and LLM. *Assignment:* Sketch a RAG system flowchart.
- Day 27: **Chunking Strategies:** Importance of document splitting ⁵. Compare token-based vs page-level ³⁰ ³¹.
- Day 28: **Chunking Lab:** Split provided docs into chunks (256–512 tokens with 15% overlap) ¹⁵. Build index (FAISS or Annoy).
- Day 29: **Metadata & Cleaning:** Tag chunks with metadata (source, date). Filter irrelevant sections (boilerplate, stop-words). *Lab:* Preprocess a raw text (e.g. PDF extract) for RAG.
- Day 30: **Quiz:** RAG concepts and chunking strategies. *Weekly Assignment:* Implement a basic retriever: given a query, return top-3 chunks.

Week 7 (9 hrs): Evaluation & Metrics

- Day 31: **BLEU Metric:** Learn BLEU (precision of n-grams) ¹. *Assignment:* Compute unigram BLEU by hand for a sample translation.
- Day 32: **ROUGE Metric:** Learn ROUGE-N/L (recall, longest common subseq) ¹⁶. *Lab:* Use Python `rouge-score` on model summaries.
- Day 33: **BERTScore:** Understand semantic scoring using BERT embeddings ². *Assignment:* Compare BLEU vs BERTScore on two paraphrases.
- Day 34: **LLM-as-Judge:** Use an LLM (e.g. GPT-3.5) to evaluate answer quality (G-Eval style). *Lab:* Write questions to check factual claims (QAG) ³².
- Day 35: **Metrics Lab:** For the RAG Q&A pipeline, evaluate performance: run BLEU/ROUGE, BERTScore, and an LLM-based judge on model answers. Discuss which captures quality best. *Weekly Assignment:* Summarize metric findings and decide final metric set for the project (e.g. accuracy + relevancy).

Week 8 (9 hrs): Inference & Streaming

- Day 36: **Deployment Basics:** Model serving options (GPU, CPU); autoscaling. Discuss latency vs throughput. *Assignment:* Estimate time to generate 200 tokens on a CPU vs GPU.
- Day 37: **Quantization & Batching:** Introduce quantization (8-bit inference) and request batching to speed up. *Lab:* Quantize a small model and compare speed.
- Day 38: **Streaming Tokens:** Importance for UX ⁶. SSE vs WebSockets for token streaming. *Lab:* Implement a simple SSE endpoint that streams tokens from an OpenAI-style completion.
- Day 39: **API Integration:** How to use streaming in client code. *Assignment:* Modify a chat UI (or terminal script) to display tokens as received.
- Day 40: **Latency Measurement:** Measure and compare token-by-token time (TTFT) with and without streaming. *Weekly Assignment:* Write a short report on how streaming improves UX and reduces waste.

Week 9 (9 hrs): AI Tools – Content and Code

- Day 41: **Google Gemini (AI Studio):** Use cases (text/image/video generation) ¹⁹. *Lab:* Call Gemini API to generate an image from text and a text summary.
- Day 42: **Lab:** Generate a piece of content (e.g. marketing blurb) with Gemini. Evaluate coherence and

style. *Assignment:* Compare outputs of Gemini vs GPT on same prompt.

- *Day 43: GitHub Copilot:* Demo code completion and chat in IDE ²⁰. *Lab:* Using Copilot, implement a coding task (e.g., binary tree). Assess helpfulness.

- *Day 44: Google Antigravity:* Introduce agentic coding. *Lab:* In Antigravity, create an agent to automatically fix failing tests in a simple code repo ²¹. Observe agent plan and actions.

- *Day 45: Tool Comparison:* Discuss pros/cons of each (cost, accessibility). *Weekly Assignment:* Document and present a case study: "Using Copilot vs manual coding – productivity trade-offs".

Week 10 (9 hrs): Agentic Workflows & Automation

- *Day 46: AI Agents Theory:* Define LLM agents (autonomous systems) ²³ ²⁴. Components: planner (LLM), tools (APIs), feedback loop.

- *Day 47: Multi-Agent Scenarios:* Orchestration of multiple LLM agents. *Assignment:* Sketch an agent workflow (e.g. travel booking agent using flight/hotel APIs).

- *Day 48: LangChain/Frameworks:* Introduction to agent libraries (LangChain Chains and Agents). *Lab:* Build a multi-step agent: e.g., user query -> search tool -> LLM -> final answer.

- *Day 49: Autonomous Task Lab:* Students use Antigravity or LangChain to create an agent with at least 2 tools (e.g., web search + JSON API) to perform a task.

- *Day 50: Project Milestone:* Each team demo's their capstone project plan and partial implementation to mentors. Mentor feedback given. *Weekly Assignment:* Refine project plan based on feedback.

Week 11 (9 hrs): Responsible AI & Red-Teaming

- *Day 51: Bias & Fairness:* Examples of model bias (gender/race stereotypes). *Assignment:* Run an LLM on biased prompts and analyze outputs.

- *Day 52: Data Privacy:* Discuss handling sensitive data in prompts (GDPR implications). *Lab:* Evaluate whether an LLM accidentally memorizes PII when over-exposed.

- *Day 53: Hallucinations:* Understand why LLMs hallucinate. Introduce factuality checkers (e.g. SelfCheckGPT). *Assignment:* Check a given answer for factual consistency using cross-references.

- *Day 54: Red-Teaming Lab:* Craft adversarial prompts to induce unsafe or biased outputs ⁴. Record vulnerabilities. *Weekly Assignment:* Submit a summary of red-team findings and mitigation suggestions.

- *Day 55: Mitigation Strategies:* Discuss content filters, human-in-loop checks. *Quiz:* Responsible AI definitions.

Week 12 (9 hrs): Capstone & Wrap-Up

- *Day 56: Project Work:* Final coding/testing on capstone tasks. Mentor Q&A.

- *Day 57: Project Work:* Finalize reports and prepare presentations.

- *Day 58: Project Presentations:* Each team demos their system, discusses results. Peer review.

- *Day 59: Feedback & Review:* Course recap; feedback surveys. Discuss certifications (Google/IBM etc).

- *Day 60: Certification & Next Steps:* Explain option for internal certification; highlight industry pathways using skills learned.

Sample Capstone Projects (Illustrative)

1. **Domain-Specific RAG Chatbot:** Develop a chatbot for medical FAQs or finance reports using RAG. *Scope:* Build a retrieval index from domain texts (e.g., medical literature), fine-tune prompts for accurate answers, and evaluate using the metrics from Week 7. *Deliverables:* Code for data pipeline (chunking, indexing), chatbot UI or API, documentation, and test QA pairs. *Datasets:* PubMed abstracts or financial filings; and sample QA (e.g. from existing datasets). *Milestones:* index ready (wk6), initial chat prototype (wk8), evaluation completed (wk10), final improvements (wk11).

2. **Content Generation Pipeline:** Automate blog writing. *Scope:* Given a topic, fetch current news (via API), generate an outline with LLM, then draft article + images. *Deliverables:* Script or app that performs search, prompt-driven content creation, and outputs a formatted draft. *Datasets/Tools:* Google News API or Reddit (web search tool), Gemini API for generation, Stable Diffusion for images (if needed). *Milestones:* search interface (wk7), content generation prompts (wk8), integration (wk9), user testing (wk11).
3. **Code Assistant Agent:** An agent that reads problem descriptions and writes+tests code. *Scope:* Use GPT-4 for code synthesis and a Python execution tool. *Deliverables:* Code for agent (e.g. LangChain agent with OpenAI Codex), example tasks (e.g. LeetCode problems), and demonstration of agent writing correct code. *Milestones:* simple prompt->code loop (wk8), add testing/checker (wk9), agent autonomy (wk10), evaluation (wk11). *Datasets:* A few programming problems with solutions for validation.

Each project includes a Gantt-timeline (Mermaid chart) for planning (see previous section). Teams are encouraged to align to real use-cases (internships often involve solving live problems for a company partner).

Hands-on Labs & Tutorials

- **Transformer & RAG:** Hugging Face transformer tutorials for training and inference; Nvidia RAG tutorials.
- **Prompt & Chatbot:** OpenAI Cookbook (prompt best practices), Azure AI prompt design guides.
- **Cost Optimization:** Use *tiktoken* for token-counting ³; experiment with prompt/payload tweaking.
- **Red-Teaming:** OpenAI's red-teaming blog ⁴; Prompt injection challenges (the Blue-Whale game).
- **Data Processing:** Use langchain's text splitters or utilities (chunking); Kaggle kernels for text pipelines.
- **Streaming Demos:** Tutorials on SSE in JavaScript/Python from the Hivenet blog ⁶.
- **Tool Demos:** Official docs for Google AI Studio/Gemini, Antigravity codelab, GitHub Copilot.

Mentorship and Ratios

- **Mentors:** For 25 students, ~3 mentors (1 academic, 2 industry) to maintain an ~8:1 ratio. Mentors hold weekly office hours and review code.
- **Guest Talks:** Industry speakers (AI engineers, data scientists) give optional talks on current trends, for motivation and networking.

Infrastructure & Budget Estimate

- **GPUs:** If on-prem, a server with 4xA100 GPUs (~\$10k capex, 3kW power) suffices. Alternatively, cloud budgets of \$3k (GPU instances) covers training/inference for projects.
- **LLM API:** Budget ~\$500 for paid APIs (e.g. for GPT-4Turbo calls in evaluation). Use free OpenAI (ChatGPT) or open models for majority to minimize cost.
- **Storage/Tools:** Minimal – use free GitHub and Google Drive. Kaggle/Azure student credits for extra.
- **Software Licenses:** Leverage free student versions (Copilot, Colab, etc.).

Certification & Partnerships

- **Certifications:** Encourage students to take *Google Cloud ML Engineer* or *Azure AI Fundamentals* after completion. Issue an internal "AIGA Internship" certificate.
- **Industry Partners:** Work with companies like Google (AI Study Jams), Microsoft (AI School), IBM (Academic Initiative) to co-develop modules or sponsor projects. Partners can provide datasets, challenge problems, or internship placements.

Sources: Content is informed by primary docs and authoritative sources: Google Cloud AI guides ¹⁴ ²⁶, OpenAI technical papers, NVIDIA and Databricks blogs on RAG ⁵, OpenAI on red teaming ⁴, and established metrics literature ¹ ². Visual aids (Fig. above) come from transformer illustrations. All citations are from official or seminal references as specified.

-
- ¹ The BLEU translation quality metric | Cloud Translation | Google Cloud Documentation
<https://docs.cloud.google.com/translate/docs/bleu-scores>
- ² ¹⁷ ¹⁸ ³² LLM Evaluation Metrics: The Ultimate LLM Evaluation Guide - Confident AI
<https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation>
- ³ Estimating The Cost of GPT Using The tiktoken Library in Python | DataCamp
<https://www.datacamp.com/pt/tutorial/estimating-cost-of-gpt-using-tiktoken-library-python>
- ⁴ cdn.openai.com
<https://cdn.openai.com/papers/openais-approach-to-external-red-teaming.pdf>
- ⁵ ¹⁵ ³⁰ ³¹ Finding the Best Chunking Strategy for Accurate AI Responses | NVIDIA Technical Blog
<https://developer.nvidia.com/blog/finding-the-best-chunking-strategy-for-accurate-ai-responses/>
- ⁶ Streaming for LLM Apps: SSE vs WebSockets | Hivenet
<https://compute.hivenet.com/post/llm-streaming-sse-websockets>
- ⁷ ⁸ ²⁹ How to Build Your First SLM. Train a small language model to sound... | by Jason Renfro | Medium
<https://medium.com/@jasonrenfro/how-to-build-your-first-slm-7a45b25d8725>
- ⁹ ¹⁰ Internship Evaluation Rubrics | PDF
<https://www.scribd.com/document/851273399/5-SEM-8-Internship-Evaluation-Rubrics>
- ¹¹ AI GPU Rental Market Trends (April 2026): Complete Industry Analysis
<https://www.thundercompute.com/blog/ai-gpu-rental-market-trends>
- ¹² krishnakumar31.files.wordpress.com
<https://krishnakumar31.files.wordpress.com/2017/09/graduates-attributes-1.pdf>
- ¹³ Transformer Explainer: LLM Transformer Model Visually Explained
<https://poloclub.github.io/transformer-explainer/>
- ¹⁴ Prompt Engineering for AI Guide | Google Cloud
<https://cloud.google.com/discover/what-is-prompt-engineering>
- ¹⁶ ROUGE (metric) - Wikipedia
[https://en.wikipedia.org/wiki/ROUGE_\(metric\)](https://en.wikipedia.org/wiki/ROUGE_(metric))
- ¹⁹ Google AI Studio | Gemini API | Google AI for Developers
<https://ai.google.dev/aistudio>
- ²⁰ ²⁷ What is GitHub Copilot? - GitHub Docs
<https://docs.github.com/en/copilot/get-started/what-is-github-copilot>

21 22 Build with Google Antigravity, our new agentic development platform - Google Developers Blog

<https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>

23 What are Agentic Workflows? | IBM

<https://www.ibm.com/think/topics/agentic-workflows>

24 Agentic AI, explained | MIT Sloan

<https://mitsloan.mit.edu/ideas-made-to-matter/agentic-ai-explained>

25 Language models are few-shot learners | OpenAI

<https://openai.com/index/language-models-are-few-shot-learners/>

26 28 Gemini API | Google AI for Developers

<https://ai.google.dev/gemini-api/docs>